

DIGILOCAL ENGINEERING & ARCHITECTURE

DigiLocal Hyperlocal Platform

Complete Backend Features & Technical Architecture Specification

Enterprise-Grade Backend Specification Document: Comprehensive blueprint detailing the 19 core platform domains, high-concurrency Node.js/Express API contracts, PostGIS spatial queries, Socket.IO real-time order alert engines, Cashfree/Razorpay payment reconciliation, double-entry Khatta ledgers, AI demand forecasting, and sub-second fleet tracking.

Platform Scope

19 Core Modules, 120+ REST/WS Endpoints, 35 Database Schemas, Multi-tenant Society Boundary Isolation.

Real-Time & Telemetry

Continuous Socket.IO order alarms, Swiggy/Zomato audio push channels, GPS live driver telemetry.

Fintech & Subscriptions

Automated vendor payouts, double-entry Khatta credit ledger, prepaid wallets, daily cron subscriptions.

AI & Multilingual Engine

Time-series demand prediction, voice-based multilingual commerce (Whisper/LLM), automated reorder alerts.

Document Version:	v3.4.0 (Production Blueprint)
System Architecture:	Node.js / Express, PostgreSQL / PostGIS / SQLite, Redis, Socket.IO
External Integrations:	Cashfree PG v3, MSG91 OTP, Firebase FCM, Expo Push, Meta WhatsApp API
Compliance Standards:	PCI-DSS Level 1 compliant gateway, strict seed data isolation, GDPR soft delete
Target Audience:	Backend Engineers, DevOps, Mobile/Frontend Leads, Product Managers, CTO

Table of Contents & Domain Index

1. Resident Profile & Account Backend Engine	Page 3	2. Society & Community Management Backend	Page 4
3. Hyperlocal Digital Store & Catalog Engine	Page 5	4. Resident Ordering & Shopping Cart Engine	Page 6
5. Real-Time Order Management & Alarm System	Page 7	6. Delivery & Fleet Management Backend	Page 8
7. Local Services & Freelancer Hub Backend	Page 9	8. Payments, Finance & Vendor Earnings Engine	Page 10
9. Digital Wallet & Society Credit (Khatta) Engine	Page 11	10. Subscription & Recurring Delivery Backend	Page 12
11. Offers, Promotions & Digital Marketing Engine	Page 13	12. WhatsApp CRM & Customer Engagement Engine	Page 14
13. AI-Powered Inventory & Demand Forecasting	Page 15	14. Regional Voice Commerce & Multilingual Engine	Page 16
15. Notifications & Multi-Channel Communications	Page 17	16. Search, Geospatial & Smart Discovery Engine	Page 18
17. Admin, Governance & Platform Management	Page 19	18. Analytics, Business Intelligence & Reporting	Page 20
19. Business Value Matrix & Architecture Blueprint	Page 21		

High-Level Architectural Topography:

The platform operates as a modular monolith / microservices-ready Node.js cluster with stateless API workers behind an Nginx reverse proxy. Redis handles transient caching, Pub/Sub channels for Socket.IO clustering, and live delivery GPS coordinates. PostgreSQL provides relational ACID integrity with PostGIS spatial indexing for high-precision society boundary containment. All background workloads (subscription cron, AI model inference batches, email queues) run via isolated async worker threads.

DOMAIN 01

Resident Profile & Account Backend Engine

CORE IDENTITY

Functional Overview & Architecture

The Resident Profile & Account engine governs customer identity, cryptographic session management, society unit tenancy verification, and persistent address books. It enforces stateless JWT authentication coupled with Redis-backed session token blacklisting and device fingerprinting.

Backend Features & Capabilities

- OTP-Based Passwordless Authentication:** Integrates with MSG91 v5 OTP service with E.164 phone normalization, configurable rate limiting (max 3 OTP requests/10 min), and cryptographic OTP simulation bypass for automated testing.
- Society & Flat Association Engine:** Binds resident profile to verified society IDs and unit/flat numbers with multi-tenant data partitioning.
- Saved Address Book:** Multi-address support (`HOME`, `WORK`, `OTHER`) with geocoded coordinates, delivery instructions, and default address flag.
- Account Lifecycle & GDPR Soft Deletion:** Self-service account deactivation and GDPR purge endpoint that retains anonymized financial audit records while scrubbing PII.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/auth/resident/otp/send	Dispatches 6-digit OTP via MSG91 SMS/WhatsApp to resident mobile number.
POST	/api/v1/auth/resident/otp/verify	Verifies OTP; returns signed JWT access token (15m) & refresh token (30d).
GET	/api/v1/resident/profile	Retrieves authenticated resident details, flat info, and active society binding.
PUT	/api/v1/resident/profile	Updates resident name, email, dietary preferences, and avatar URL.
GET	/api/v1/resident/addresses	Fetches all saved delivery addresses with coordinates and delivery notes.
POST	/api/v1/resident/addresses	Creates a new delivery address; validates coordinate proximity to society boundary.
DELETE	/api/v1/resident/account	Initiates account soft-delete, revokes active refresh tokens, and purges sessions.

Data Entities & Schema Design: `resident_users` (id, phone, full_name, email, society_id, flat_number, tower_block, is_verified, is_blocked, created_at), `resident_addresses` (id, user_id, label, address_line, flat_no, latitude, longitude, is_default), `resident_sessions` (id, user_id, refresh_token_hash, device_info, expires_at).

DOMAIN 02

Society & Community Management Backend

HYPERLOCAL GEOFENCE

Functional Overview & Architecture

The Society Management engine enforces geofenced hyperlocal boundaries, restricting marketplace discovery, vendor catalog visibility, and delivery dispatching to approved residential gated communities.

Backend Features & Capabilities

- Geofenced Society Indexing:** Employs PostGIS polygons and spatial bounding boxes (`ST\_Contains`, `ST\_DWithin`) to automatically match resident coordinates to their respective registered society.
- Vendor-to-Society Mapping Matrix:** Multi-tenant isolation ensuring only merchants explicitly approved by a society or designated as "Universal Hyperlocal" appear in that society's storefront.
- Society-Level Delivery Rules:** Dedicated configuration per society: security gate verification protocols, delivery timing cut-offs, entry gate drop-off versus doorstep delivery flags.
- Community Announcements & Bulletins:** High-priority administrative notices and society-specific promotions with scheduled publication windows.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/societies	Public directory listing active societies filtered by city, pincode, or name.
GET	/api/v1/societies/nearby	Spatial query resolving closest societies based on resident GPS lat/long coords.
GET	/api/v1/societies/:id/vendors	Lists all verified and operational merchants mapped to the specified society.
GET	/api/v1/societies/:id/announcements	Fetches active community announcements, notices, and society-wide discount campaigns.
POST	/api/v1/admin/societies	[Admin] Registers a new society with geofence polygon coords, gate rules, and unit count.
PUT	/api/v1/admin/societies/:id/vendor-map	[Admin/Society RWA] Binds or revokes vendor operating permissions for this community.

Data Entities & Schema Design: **societies** (id, name, registration_code, address, city, pincode, boundary_polygon, total_units, security_gate_policy, is_active), **society_vendor_mappings** (id, society_id, vendor_id, approval_status, commission_rate_override, is_exclusive), **society_announcements** (id, society_id, title, body, banner_url, valid_until).

DOMAIN 03

Hyperlocal Digital Store & Catalog Engine

MERCHANT STOREFRONT

Functional Overview & Architecture

The Digital Store & Catalog engine provides merchants with instant digital storefront configuration, SKU management, multi-tier categorization, variant matrix definitions, and sub-millisecond stock availability toggles.

Backend Features & Capabilities

- Storefront Configuration & Dynamic Timings:** Daily opening/closing hours, lunch breaks, holiday schedules, and automated "Open/Closed" status computation based on server IST time.
- Hierarchical Catalog Management:** Categories, subcategories, product attributes, custom unit measures (`kg`, `gram`, `piece`, `liter`, `pack`), and multi-resolution image asset optimization.
- Instant Stock & Availability Switch:** Real-time toggle (`in\_stock: boolean`, `stock\_qty: integer`) with automated eviction of cached catalog entries in Redis.
- MOV & Delivery Policy Customization:** Vendor-specific Minimum Order Value (MOV), flat delivery fees, free delivery thresholds, and maximum allowed order units per line item.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/vendor/store-profile	Retrieves vendor store details, operating hours, delivery rules, and logo URLs.
PUT	/api/v1/vendor/store-profile	Updates store description, contact numbers, delivery charges, and MOV limits.
POST	/api/v1/vendor/store/logo	Multipart image upload; processes image and generates CDN-hosted web-optimized URLs.
GET	/api/v1/vendor/products	Paginated merchant catalog with filters for category, stock status, and text search.
POST	/api/v1/vendor/products	Creates new product SKU with pricing, discount %, variants, GST rate, and stock qty.
PATCH	/api/v1/vendor/products/:id/stock	Atomic toggle for product availability state; emits cache invalidation event.
GET	/api/v1/public/stores/:id/catalog	High-speed public catalog endpoint for resident apps backed by Redis caching.

Data Entities & Schema Design: **vendors** (id, store_name, owner_name, email, phone, gst_number, logo_url, banner_url, opening_time, closing_time, min_order_value, delivery_charge, status), **products** (id, vendor_id, category_id, name, description, unit, base_price, sale_price, stock_quantity, is_available, image_url, hsn_code, tax_rate).

DOMAIN 04

Resident Ordering & Shopping Cart Engine

TRANSACTION PROCESSING

Functional Overview & Architecture

The Resident Ordering engine handles atomic shopping cart validation, complex bill calculations (line items, vendor taxes, delivery fees, dynamic coupon discounts), inventory lockouts, and order placement pipelines.

Backend Features & Capabilities

- Cart Price & Availability Validation:** Validates item pricing against live database values, rejects stale client prices, verifies stock availability, and checks vendor MOV requirements.
- Tax & Delivery Charge Computation:** Calculates line-item GST, society-specific delivery surcharges, platform service fees, and validates coupon applicability before locking total bill.
- Atomic Order Placement:** Uses SQL database transactions with row-level locking (`SELECT ... FOR UPDATE`) to prevent inventory overselling during peak concurrent flash orders.
- Order History & 1-Click Reorder:** Comprehensive order audit history with line-item breakdowns, invoice generation, real-time tracking states, and 1-click cart rehydration for past orders.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/cart/validate	Validates client cart items, verifies vendor open hours, stock qty, and MOV.
POST	/api/v1/cart/calculate-total	Computes item subtotal, applicable taxes, delivery fee, and discount deductions.
POST	/api/v1/orders/checkout	Creates order intent, holds inventory lock for 10 minutes, returns checkout session.
POST	/api/v1/orders/place	Finalizes order placement for COD, Wallet, or verified Cashfree payment session.
GET	/api/v1/resident/orders	Retrieves paginated historical orders for authenticated resident.
GET	/api/v1/resident/orders/:id	Fetches order breakdown, itemized receipt, delivery address, and live status.
POST	/api/v1/resident/orders/:id/cancel	Cancels order if not yet accepted/dispatched; triggers automated refund flow.

Data Entities & Schema Design: `orders` (id, order_number, user_id, vendor_id, society_id, total_amount, subtotal, tax_amount, delivery_fee, discount_amount, payment_method, payment_status, order_status, created_at), `order_items` (id, order_id, product_id, product_name, unit_price, quantity, total_price).

DOMAIN 05

Real-Time Order Management & Alarm System

SWIGGY/ZOMATO ALERT ENGINE

Functional Overview & Architecture

The Real-Time Order Management system features a multi-tiered order alert pipeline emulating Swiggy/Zomato merchant devices. It synchronizes WebSocket room events, high-priority Firebase push notifications with custom audio chimes, and automatic repetition loops until merchant acknowledgment.

Backend Features & Capabilities

- WebSocket Event Multiplexing:** Socket.IO server isolates vendors into dedicated rooms (``vendor_{vendor_id}``) and broadcasts instant ``NEW_ORDER_ALERT`` payloads upon order creation.
- Repetitive Audio Chime Notification Loop:** When a new order arrives, the server dispatches a high-priority FCM payload targeting the Android channel ``order_alerts_channel`` configured with sound ``order_alert_chime.wav``. A background worker repeats alerts every 30s until acknowledged.
- Order Status Transition Pipeline:** Strictly validated status finite state machine: ``PLACED`` → ``ACCEPTED`` → ``PREPARING`` → ``READY_FOR_PICKUP`` → ``OUT_FOR_DELIVERY`` → ``DELIVERED``.
- Live Resident Status Broadcast:** Every status transition triggers an instantaneous WebSocket push to the resident's client connection, updating their active tracking interface.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/vendor/orders/active	Fetches all pending, accepted, and in-preparation orders for the vendor panel.
POST	/api/v1/vendor/orders/:id/acknowledge	Acknowledges order; stops repetitive audio alarm loop and marks merchant alerted.
PATCH	/api/v1/vendor/orders/:id/status	Advances order status (<code>`ACCEPTED`</code> , <code>`PREPARING`</code> , <code>`READY`</code> , <code>`DISPATCHED`</code> , <code>`REJECTED`</code>).
POST	/api/v1/vendor/device-token	Registers FCM/Expo push notification token with device audio capabilities.
WS	emit: NEW_ORDER_ALERT	Socket.IO push payload containing order ID, total bill, customer name, and items.
WS	emit: ORDER_STATUS_CHANGED	Broadcasts live state updates to resident client tracking view.

Data Entities & Schema Design: `order_status_history` (id, order_id, previous_status, new_status, updated_by_user_id, updated_by_role, notes, timestamp), `vendor_device_tokens` (id, vendor_id, push_token, device_platform, is_active, last_seen_at), `alarm_acknowledgments` (id, order_id, vendor_id, acknowledged_at, latency_seconds).

DOMAIN 06

Delivery & Fleet Management Backend

LOGISTICS & GPS TRACKING

Functional Overview & Architecture

The Delivery & Fleet Management backend powers dynamic delivery assignment, driver dispatching, turn-by-turn navigation telemetry, sub-second GPS coordinate ingestion via Redis Geospatial, and cryptographically verified OTP delivery handshakes.

Backend Features & Capabilities

- Smart Rider Allocation Engine:** Automatically evaluates available delivery fleet members based on proximity to the vendor store and current active assignment load.
- High-Frequency GPS Ingestion Pipeline:** Ingests rider latitude/longitude pings via lightweight UDP/HTTP endpoints into Redis GEO sets (`GEOADD delivery\_riders`), enabling live map tracking without overloading relational databases.
- Real-Time Resident Tracking Stream:** Pipes rider coordinate deltas over WebSockets directly to the resident order tracking map, calculating dynamic ETA based on distance and traffic.
- Proof of Delivery (POD) & OTP Verification:** Secure 4-digit handover OTP generated on the resident's app that the driver must submit to mark the order `DELIVERED`.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/delivery/auth/login	Authenticates delivery personnel; establishes driver session and active duty status.
GET	/api/v1/delivery/assigned-orders	Lists active assigned deliveries with pickup store address and resident flat info.
PATCH	/api/v1/delivery/orders/:id/accept	Driver accepts order assignment; notifies vendor and resident.
POST	/api/v1/delivery/telemetry/location	Pushes high-frequency GPS ping (lat, lng, speed, heading) into Redis GEO store.
GET	/api/v1/orders/:id/live-tracking	Returns current delivery rider location, polyline route, and updated ETA.
POST	/api/v1/delivery/orders/:id/verify-otp	Verifies resident delivery OTP; marks delivery completed and credits driver payout.

Data Entities & Schema Design: `delivery_riders` (id, full_name, phone, vehicle_type, license_plate, is_online, current_lat, current_lng, rating), `delivery_assignments` (id, order_id, rider_id, assigned_at, accepted_at, picked_up_at, delivered_at, delivery_otp_hash, delivery_proof_image).

DOMAIN 07

Local Services & Freelancer Hub Backend

ON-DEMAND SERVICES

Functional Overview & Architecture

The Local Services & Freelancer Hub expands DigiLocal beyond physical goods into hyper-local on-demand home services (electricians, plumbers, house cleaners, appliance repair, tutoring, pet care) within residential societies.

Backend Features & Capabilities

- Provider KYC & Profile Verification:** Admin verification workflows for service freelancers with identity proof, background checks, skill certifications, and society clearance.
- Dynamic Rate Cards & Service Catalogs:** Configurable service tariffs (fixed price inspection fee, hourly rates, or task-based pricing) with custom service area radius.
- Appointment Scheduling Engine:** Slot availability calendar, booking time-slot reservations, conflict detection, and calendar synchronization.
- Two-Way Communication & Review Matrix:** In-app messaging between resident and service provider, combined with post-service ratings and review aggregation.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/services/categories	Fetches all active service categories (e.g. Electrical, Plumbing, Cleaning).
GET	/api/v1/services/providers	Lists verified providers operating within resident's society with ratings and pricing.
POST	/api/v1/services/bookings	Creates a service request with selected slot, problem description, and address.
PATCH	/api/v1/services/bookings/:id/status	Updates booking state ('REQUESTED', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED').
POST	/api/v1/services/bookings/:id/reviews	Submits customer rating (1-5 stars) and feedback; triggers rating recalculation.

Data Entities & Schema Design: `service_providers` (id, full_name, phone, category_id, experience_years, bio, hourly_rate, is_verified, average_rating), `service_bookings` (id, user_id, provider_id, scheduled_date, scheduled_slot, status, estimated_cost, final_cost, payment_status).

DOMAIN 08

Payments, Finance & Vendor Earnings Engine

FINTECH & SETTLEMENT

Functional Overview & Architecture

The Payments, Finance & Earnings engine acts as DigiLocal's financial ledger. It coordinates Cashfree PG v3 payment gateway sessions, webhook signature validations, merchant commission deductions, and automated payout settlements.

Backend Features & Capabilities

- Cashfree PG v3 Gateway Integration:** Secure checkout session generation (`createPaymentSession``), handling UPI Intent, NetBanking, and credit/debit cards with automated payment status verification.
- HMAC SHA256 Webhook Verification:** Cryptographic verification of incoming webhook payloads to eliminate man-in-the-middle payment spoofing.
- Vendor Earnings Ledger & Commission Deductions:** Real-time financial accounting logging gross sales, platform commissions (e.g. 5%), delivery fee allocations, and net payable balances.
- Automated Bank Payouts:** Automated settlement disbursements to vendor bank accounts via Cashfree Payouts API / IMPS transfer, with downloadable tax invoices and settlement reconciliation reports.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/payments/cashfree/session	Creates Cashfree PG checkout order session; returns encrypted payment session ID.
POST	/api/v1/payments/cashfree/webhook	Public webhook listener; verifies HMAC signature and updates order payment status.
GET	/api/v1/vendor/finance/earnings	Summary of total gross sales, platform commission fees, and available payout balance.
GET	/api/v1/vendor/finance/settlements	Historical list of automated bank transfers, IMPS UTR references, and dates.
POST	/api/v1/admin/finance/payouts/trigger	[Admin] Initiates bulk automated payout disbursement batch for eligible merchants.

Data Entities & Schema Design: `payment_transactions` (id, order_id, gateway, transaction_ref, amount, currency, status, payment_method, signature_verified, created_at), `vendor_settlements` (id, vendor_id, amount, platform_fee, net_settled, bank_utr, settlement_status, processed_at).

DOMAIN 09

Digital Wallet & Society Credit (Khatta) Engine

LEDGER & KHATTA (UDHAR)

Functional Overview & Architecture

The Digital Wallet & Khatta system implements a double-entry ledger architecture. It manages prepaid resident balances, instant wallet checkouts, and digitizes traditional neighborhood credit books ("Udhar / Khatta") with credit limits and automated payment reconciliation.

Backend Features & Capabilities

- Prepaid Resident Wallet:** Instant 1-click checkouts, instant refund crediting for cancelled orders, and wallet top-up via UPI/Payment Gateway.
- Double-Entry Ledger Integrity:** Every credit and debit transaction is recorded as balanced pairs in an immutable ledger table with cryptographic hash chaining to prevent tampering.
- Merchant Khatta / Udhar Ledger:** Allows trusted society residents to purchase goods on credit within pre-configured credit limits set by the vendor or society association.
- Credit Settlement & Due Reminders:** Automated calculation of outstanding balances, WhatsApp settlement reminders, and partial or full payment reconciliation.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/resident/wallet	Returns active wallet balance, promotional bonus balance, and transaction history.
POST	/api/v1/resident/wallet/topup	Creates a top-up payment session; credits wallet balance on gateway confirmation.
GET	/api/v1/vendor/khatta/customers	Lists customers with active credit accounts, credit limits, and outstanding balances.
POST	/api/v1/vendor/khatta/entry	Records an Udhar/Credit purchase or offline cash payment against customer ledger.
POST	/api/v1/vendor/khatta/send-reminder	Dispatches automated WhatsApp/SMS payment reminder with UPI payment link.

Data Entities & Schema Design: `resident_wallets` (id, user_id, current_balance, locked_balance, currency, is_active), `wallet_ledger_entries` (id, wallet_id, type [CREDIT/DEBIT], amount, reference_type [ORDER/TOPUP/REFUND], reference_id, balance_after), `khatta_accounts` (id, vendor_id, user_id, credit_limit, outstanding_balance, status), `khatta_ledger` (id, khatta_id, transaction_type, amount, balance_after, note).

DOMAIN 10

Subscription & Recurring Delivery Backend

AUTOMATED DAILY SUBSCRIPTIONS

Functional Overview & Architecture

The Subscription engine automates daily and periodic delivery essentials (milk packets, morning newspapers, fresh bread, bottled drinking water, eggs) to residential society doors through cron-orchestrated order dispatchers.

Backend Features & Capabilities

- Flexible Recurring Cadence Engine:** Supports daily, alternate days, weekdays only, weekend only, and custom weekly day matrices (e.g. Mon-Wed-Fri).
- Vacation Pause & Instant Resume:** Residents can pause subscriptions with a single tap before the 10:00 PM cut-off time without incurring charges during vacations.
- Automated Nightly Order Generation Cron:** A scheduled background worker executes daily at 03:00 AM IST (``0 3 * * *``), queries active subscriptions, verifies wallet balances or flags COD, and spawns finalized morning delivery orders for vendors.
- Subscription Health & Predictive Delivery:** Tracks recurring delivery history, missed delivery alerts, and upcoming 7-day schedule previews for residents.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/subscriptions/catalog	Lists eligible recurring products (milk, dairy, water, bread, newspapers).
POST	/api/v1/subscriptions	Creates a recurring subscription plan with schedule frequency, quantity, and start date.
PATCH	/api/v1/subscriptions/:id/pause	Sets pause window (<code>`start_date`</code> to <code>`end_date`</code>) preventing order generation.
PATCH	/api/v1/subscriptions/:id/resume	Re-activates subscription ahead of schedule; resumes daily delivery cycle.
GET	/api/v1/subscriptions/upcoming	Generates a 7-day visual calendar forecast of expected daily deliveries and costs.

Data Entities & Schema Design: `subscriptions` (id, user_id, vendor_id, product_id, quantity, frequency [DAILY/WEEKDAYS/CUSTOM], custom_days_json, start_date, end_date, status [ACTIVE/PAUSED/CANCELLED], billing_type), `subscription_deliveries` (id, subscription_id, delivery_date, generated_order_id, status, notes).

DOMAIN 11

Offers, Promotions & Digital Marketing Engine

GROWTH & MARKETING

Functional Overview & Architecture

The Promotions & Marketing engine drives consumer demand and merchant growth via dynamic rule-based coupon validation, society-exclusive flash sales, homepage hero banner carousels, and targeted promotional broadcasts.

Backend Features & Capabilities

- Rule-Based Coupon Code Engine:** Validates complex eligibility criteria: minimum cart value, maximum discount cap, user usage count (e.g. `FIRSTORDER`), and society-restricted promo codes.
- Hero Banners & Dynamic Placement:** Admin/Merchant banner manager supporting placements (`HOME\_HERO`, `POPUP`, `CATEGORY\_HEADER`), target URLs, display ordering, and automated expiry.
- Society-Specific Flash Deals:** Limited-time price drops restricted to specific residential communities, creating high-conversion community shopping frenzies.
- Personalized Resident Offers:** Recommendation worker identifies dormant or frequent users and auto-issues targeted discount tokens.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/promotions/coupons/validate	Tests coupon code against cart payload; returns calculated discount or error reason.
GET	/api/v1/promotions/banners	Fetches active marketing banners filtered by resident society and device type.
POST	/api/v1/admin/promotions/campaigns	[Admin] Launches marketing campaign with schedule dates, target societies, and assets.
GET	/api/v1/vendor/promotions	Allows merchants to review redemption metrics, promo codes, and discount costs.

Data Entities & Schema Design: `coupons` (id, code, description, discount_type [PERCENTAGE/FLAT], discount_value, min_cart_value, max_discount, usage_limit_per_user, total_usage_limit, valid_from, valid_to, target_society_id, is_active), `promotional_banners` (id, title, image_url, target_type, target_id, placement, sort_order, is_active).

DOMAIN 12

WhatsApp CRM & Customer Engagement Engine

META CLOUD WHATSAPP

Functional Overview & Architecture

The WhatsApp CRM integrates directly with the Meta WhatsApp Cloud API and webhooks. It enables automated transactional dispatches, two-way interactive chatbots, direct merchant-to-customer click-to-chat deeplinks, and lightweight merchant CRM segmentation.

Backend Features & Capabilities

- Transactional WhatsApp Dispatches:** Dispatches rich WhatsApp template messages for order confirmation, out-for-delivery alerts, bill PDFs, and daily milk delivery summaries.
- Direct 1-Click Merchant Chat:** Generates pre-filled `https://wa.me/` deeplinks with formatted order summaries and items, allowing vendors to message customers without manual typing.
- Vendor CRM & Customer Segmentation:** Aggregates customer profiles for merchants: lifetime spend, order frequency, favorite products, and overdue credit balances.
- Follow-Up Tasks & Customer Notes:** Enables vendors to save internal notes against resident profiles (e.g. "prefers deliveries before 8 AM") and schedule follow-ups.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/integrations/whatsapp/webhook	Meta Cloud API webhook listener for incoming messages, delivery receipts, and reads.
GET	/api/v1/vendor/orders/:id/wa-link	Generates structured WhatsApp click-to-chat URL with complete itemized order details.
GET	/api/v1/vendor/crm/customers	Lists vendor's top customers with order frequency, total spend, and last order date.
POST	/api/v1/vendor/crm/customers/:id/notes	Saves private operational notes or preferences for a specific customer.

Data Entities & Schema Design: `whatsapp_message_logs` (id, recipient_phone, template_name, message_status [SENT/DELIVERED/READ/FAILED], external_message_id, sent_at), `vendor_crm_notes` (id, vendor_id, customer_user_id, note_text, created_at).

DOMAIN 13

AI-Powered Inventory & Demand Forecasting Engine

PREDICTIVE ANALYTICS

Functional Overview & Architecture

The AI-Powered Inventory engine utilizes time-series machine learning models and heuristic aggregation workers to analyze 90-day purchase patterns, seasonal festival demand spikes, and daily society consumption habits to optimize merchant stock levels.

Backend Features & Capabilities

- Predictive Demand Forecasting:** Processes historical daily velocity, weather data, and society demographic makeup to forecast SKU unit demand for the upcoming 7 and 14 days.
- Smart Automated Reorder Recommendations:** Calculates safety stock thresholds, lead-time buffers, and alerts vendors when SKUs approach critical depletion levels.
- Merchandise Velocity Classification:** Classifies store inventory into Fast-Moving Consumer Goods (FMCG), Slow-Moving, and High-Margin items to guide stocking decisions.
- Out-of-Stock Prevention Alerts:** Automated background worker monitors current order run-rate against remaining inventory and triggers push notifications before stockouts occur.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/vendor/ai/demand-forecast	Returns forecasted unit sales for top 20 SKUs for the next 7 days with confidence score.
GET	/api/v1/vendor/ai/reorder-advice	Lists recommended purchase orders with suggested reorder quantities and supplier links.
GET	/api/v1/vendor/ai/product-velocity	Categorizes catalog into Fast-Moving, Steady, and Dead-Stock items.
POST	/api/v1/vendor/inventory/bulk-adjust	Atomically updates stock levels across multiple SKUs following warehouse delivery.

Data Entities & Schema Design: `inventory_snapshots` (id, vendor_id, product_id, snapshot_date, opening_stock, units_sold, closing_stock, stockout_flag), `ai_demand_predictions` (id, vendor_id, product_id, forecast_date, predicted_demand_units, confidence_interval, generated_at).

DOMAIN 14

Regional Voice Commerce & Multilingual Experience

VOICE AI & LOCALIZATION

Functional Overview & Architecture

The Voice Commerce & Multilingual engine lowers digital barriers for elderly residents, domestic staff, and non-English-speaking users by supporting regional Indian languages, speech-to-text NLP parsing, and automated phonetic catalog matching.

Backend Features & Capabilities

- Multi-Language Localization Engine:** Dynamic internationalization (i18n) supporting English, Hindi, Marathi, Gujarati, Kannada, Tamil, and Telugu across UI copy, categories, and receipts.
- Voice Audio Ingestion & STT Pipeline:** Ingests short audio clips (`.m4a`, `.wav`, `.ogg`), transcribes via OpenAI Whisper / Google Speech API, and extracts shopping intent via an NLP entity parser.
- Fuzzy Phonetic SKU Resolution:** Maps regional spoken phrases (e.g. *"do packet amul taaza doodh aur ek bread"*) to exact catalog product IDs and quantities using Levenshtein distance and embedding search.
- Voice-Assisted 1-Tap Cart Injection:** Automatically populates the resident's cart with parsed voice items, returning structured confirmation dialogs for audio confirmation.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
POST	/api/v1/voice/parse-intent	Ingests raw voice audio file; transcribes and returns extracted product names & quantities.
POST	/api/v1/voice/smart-cart-populate	Resolves voice intent against society vendor catalog; injects items directly into user cart.
GET	/api/v1/i18n/translations	Fetches localized dictionary strings for the client's selected regional language.

Data Entities & Schema Design: `voice_query_logs` (id, user_id, raw_audio_url, transcribed_text, language_code, detected_intent, matched_product_ids_json, execution_ms), `product_i18n` (id, product_id, language_code, localized_name, localized_desc).

DOMAIN 15

Notifications & Multi-Channel Communications Engine

NOTIFICATION MATRIX

Functional Overview & Architecture

The Notifications engine acts as an omni-channel dispatch router. It coordinates Push Notifications (Firebase FCM, Expo Push), SMS/WhatsApp (MSG91), Transactional HTML Emails (Nodemailer), and In-App notification inboxes with unified deduplication guards.

Backend Features & Capabilities

- Multi-Channel Priority Dispatch Router:** Directs critical alerts (Order Placed, OTP) through high-deliverability SMS/WhatsApp and push notifications simultaneously.
- Notification Deduplication & Rate Limiting:** Redis key caching prevents phantom double-pushes when webhooks or retries fire concurrently.
- Transactional Email Queue:** Asynchronous background email queue (`src/services/emailQueue.js`) prevents SMTP socket latency from blocking HTTP API response loops.
- Persistent In-App Notification Center:** Resident and merchant in-app notification center with read/unread flags, deep-linking targets, and push badge counts.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/notifications	Fetches paginated user notifications with read status, timestamps, and action URLs.
PATCH	/api/v1/notifications/:id/read	Marks individual notification as read; decrements unread badge count.
PATCH	/api/v1/notifications/mark-all-read	Marks all pending alerts as read for the authenticated session.
POST	/api/v1/admin/notifications/broadcast	[Admin] Broadcasts urgent society-wide push alert to all registered residents.

Data Entities & Schema Design: `notifications_inbox` (id, recipient_id, recipient_role [USER/VENDOR/DRIVER], title, body, icon, deep_link, is_read, created_at), `notification_dispatch_logs` (id, channel [FCM/SMS/EMAIL/WA], external_id, status, latency_ms).

DOMAIN 16

Search, Geospatial & Smart Discovery Engine

FULL-TEXT & DISCOVERY

Functional Overview & Architecture

The Search & Smart Discovery engine indexes products, stores, and services using PostgreSQL full-text search (`tsvector` / `pg_trgm``) and geospatial bounding boxes. It delivers typo-tolerant instant search and personalized product recommendations.

Backend Features & Capabilities

- Typo-Tolerant Trigram Search:** Handles spelling variations (e.g. "biscuit", "biskit", "parle-g") across thousands of SKUs in sub-15ms query times.
- Geospatial Store Proximity Search:** Filters and sorts stores based on radial distance from resident coordinates using PostGIS spherical distance functions.
- Collaborative Recommendation Engine:** Aggregates "Frequently Bought Together" items and trending purchases specific to the resident's residential society.
- Zero-Result Fallback & Search Telemetry:** Logs search queries with zero matching products to inform vendors of unmet resident demand in their society.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/discovery/search	Unified search returning matching products, active stores, and available services.
GET	/api/v1/discovery/trending	Fetches top 10 trending items ordered within the resident's specific housing society.
GET	/api/v1/discovery/recommended	Personalized product recommendations based on user's past 30-day order history.

Data Entities & Schema Design: `search_analytics_logs` (id, user_id, society_id, query_term, matched_results_count, clicked_product_id, searched_at), `product_recommendation_cache` (id, product_id, recommended_product_id, score).

DOMAIN 17

Admin, Governance & Platform Management Engine

SUPER ADMIN & RBAC

Functional Overview & Architecture

The Admin & Governance platform provides administrators with centralized control, sub-admin role delegation, merchant KYC validation, user dispute mediation, and immutable audit logging for regulatory compliance.

Backend Features & Capabilities

- Granular Role-Based Access Control (RBAC):** Hierarchical sub-admin permissions ('MANAGE_USERS', 'MANAGE_VENDORS', 'FINANCIALS', 'SUPPORT', 'SETTINGS', 'CMS', 'AUDIT_LOGS').
- Vendor Onboarding & Verification Workflow:** Two-phase merchant review: document verification (GSTIN, FSSAI, PAN), bank detail approval, and 1-Year subscription activation.
- Order & Delivery Escalation Console:** Real-time oversight over all platform orders; ability to cancel stuck orders, reassign riders, or initiate instant refunds.
- Immutable Audit Logging Trail:** Automatically records every administrative action, IP address, timestamp, and entity diff for governance and security auditing.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/admin/dashboard/metrics	Platform-wide telemetry: total GMV, active users, live vendors, and order counts.
GET	/api/v1/admin/vendors/pending	Lists merchant onboarding applications awaiting administrative review and KYC approval.
POST	/api/v1/admin/vendors/:id/approve	Approves vendor; creates 1-year active subscription and sends welcome email.
POST	/api/v1/admin/vendors/:id/reject	Rejects application with custom reason; triggers rejection email notification.
GET	/api/v1/admin/audit-logs	Fetches paginated audit logs with search by admin user, entity type, and date range.
GET	/api/v1/admin/audit-logs/export	Exports filtered audit logs into CSV/JSON for compliance review.

Data Entities & Schema Design: `admin_users` (id, username, email, password_hash, role [SUPER_ADMIN/SUB_ADMIN], permissions_array, is_active), `system_audit_logs` (id, actor_id, actor_role, action, target_entity, target_id, ip_address, user_agent, diff_payload, created_at).

DOMAIN 18

Analytics, Business Intelligence & Reporting Engine

BI & DATA PIPELINES

Functional Overview & Architecture

The Analytics & Business Intelligence platform transforms raw transactional streams into actionable insights. It utilizes nightly rollup cron jobs, materialized views, and OLAP data structures to compute KPIs without impacting operational OLTP performance.

Backend Features & Capabilities

- Vendor Business Intelligence:** Sales trends, gross revenue curves, average order value (AOV), repeat customer retention rates, and peak ordering time windows.
- Society-Level Commerce Analytics:** Society gross consumption volume, resident penetration rate, top-performing product categories, and vendor order fulfillment rates.
- Platform Telemetry & Executive GMV Dashboards:** Gross Merchandise Value (GMV), platform net revenue take-rate, customer acquisition trends, and logistics SLA compliance.
- Automated Report Export Pipeline:** Generates formatted CSV and PDF financial and operational reports with asynchronous background job generation.

API Endpoint Contracts

Method	Endpoint Path	Functional Description & Auth
GET	/api/v1/vendor/analytics/sales-overview	Daily, weekly, and monthly revenue aggregates, order volume, and average basket size.
GET	/api/v1/vendor/analytics/top-products	Ranks top-selling products by quantity sold and gross revenue contribution.
GET	/api/v1/admin/analytics/platform-gmv	Executive revenue metrics, platform commission collections, and growth percentages.
GET	/api/v1/admin/analytics/society-metrics	Comparative analytics ranking societies by order frequency and active spending residents.

Data Entities & Schema Design: `daily_vendor_metrics` (id, vendor_id, metric_date, total_orders, completed_orders, cancelled_orders, gross_revenue, net_earnings, unique_customers), `daily_society_metrics` (id, society_id, metric_date, active_users_count, total_orders, total_spend).

DOMAIN 19

Business Value Matrix & Architecture Blueprint

PLATFORM VALUE & TOPOLOGY

Stakeholder Value Realization Matrix

Stakeholder	Core Value Proposition	Backend Architectural Enablers
Residents	Doorstep hyperlocal shopping, 15-30 min deliveries, trusted society vendors, recurring milk/groceries, multi-language voice ordering.	Sub-second catalog caching, PostGIS geofence isolation, cron subscription generation, double-entry wallet balance.
Merchants / Vendors	Zero-setup digital store, access to captive gated community buyers, Swiggy-style loud audio alarms, automated bank payouts, Khatta credit tracking.	FCM max-priority push audio channels, Socket.IO rooms, Cashfree auto-settlement, immutable double-entry credit ledger.
Society RWAs	Organized community marketplace, vetted external vendor access, security gate verification compliance, community announcements.	Multi-tenant society boundary policies, security gate delivery protocol flags, RWA admin audit logs.
Delivery Fleet	Optimized route navigation, fair automated dispatching, verified OTP delivery proof, transparent earnings tracking.	Redis GEO spatial coordinate tracking, real-time ETA calculation, cryptographic POD OTP handshake.
Platform Operators	Scalable multi-society marketplace, recurring subscription revenue, commission take-rates, complete audit & governance controls.	Modular monolith / microservice architecture, OpenAPI 3.1.0 specifications, strict database seed isolation, observability probes.

Core Backend Architecture & Technical Specifications

Layer / Subsystem	Technology Selection	Key Implementation Attributes
Runtime & Framework	Node.js LTS, Express 4.x	Stateless async/await request handlers, Winston structured logging.
Database & Spatial	PostgreSQL 15+ with PostGIS / SQLite	Unified abstraction layer (<code>src/models/db.js</code>), connection pooling, ACID transactions.
Caching & In-Memory	Redis 7.x	Sub-millisecond catalog caching, TTL key deduplication, Redis GEO for driver GPS.
Real-Time Engine	Socket.IO 4.x with Redis Adapter	Vendor notification rooms (<code>vendor_{id}</code>), resident live tracking rooms.
Payments & Fintech	Cashfree PG v3 API & Payouts Engine	HMAC SHA256 webhook signatures, automated refunds, bank settlement ledger.
SMS & WhatsApp OTP	MSG91 v5 OTP API	E.164 phone formatting, developer simulation fallback (<code>123456</code>).
Push Notifications	Firebase Admin SDK (FCM) & Expo Push	Custom sound channel (<code>order_alert_chime.wav</code>), Android max priority.
Background Workers	node-cron, Async Email Queue	Nightly 3 AM subscription order runner, 9 AM expiry warnings, Nodemailer.
Observability & Probes	Docker, Kubernetes Health Probes	<code>/health</code> (system ping), <code>/health/live</code> , <code>/health/ready</code> , memory & uptime telemetry.

Production Data & Testing Isolation Rule: All automated test suites and smoke runners strictly adhere to zero-mutation policies on production and seed records. Ephemeral dummy data with unique timestamped UUIDs is provisioned and cleaned up in `afterAll` test hooks.